

# UD3 – Seguridad lógica: Criptografía y sistemas de identificación

MEDAC
Instituto Oficial de Formación Profesional

# Objetivos

1. Describir las diferencias que existen entre la criptografía simétrica y la criptografía asimétrica.
2. Reconocer los principales algoritmos de clave pública y de clave privada.
3. Usas criptografía simétrica y asimétrica
4. Reconocer las ventajas y aplicaciones de la función resumen hash.
5. Conocer los principales sistemas de identificación y el uso de listas de control de acceso.



# Preparación de entorno de aprendizaje

MEDAC
Instituto Oficial de Formación Profesional

# Máquinas Virtuales

- Es un Sistema operativo que ejecutamos dentro de nuestro Sistema operativo principal (host)

- Características:
  - Comparten recursos de HW (RAM, disco, CPU) con la máquina host de manera controlada. Le indicaremos cuanta cantidad de disco y memoria queremos que use como máximo.
  - Se comportan como entornos aislados. El SW que instalemos en una máquina virtual no afecta a nuestro Sistema Operativo principal. Permite analizar SW malicioso sin infectar nuestra máquina principal

# SW de Virtualización

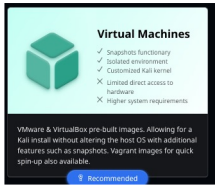
- Necesitamos un SW de virtualización para crear máquinas Virtuales
- Hay dos SW principales en el mercado: VirtualBox y VMWare
- Usaremos VirtualBox -> (Instalar)

# Instalando Kali Linux

- Kali Linux: Esta máquina la usaremos como máquina del atacante cuando realizamos un proceso de Hacking Ético

- Descarga en : https://www.kali.org/get-kali/#kali-platforms
  (en buscador: Kali Linux download)

- Tenemos varias opciones de descarga. Hay dos principales:
  - Descargar una imagen .iso. Nos sirve tanto para instalar en máquina virtual como en máquina física
  - Importar una de las máquinas virtuales que se nos proporciona ya operativa. **Esta es la opción que elegiremos**



# Instalando Kali Linux

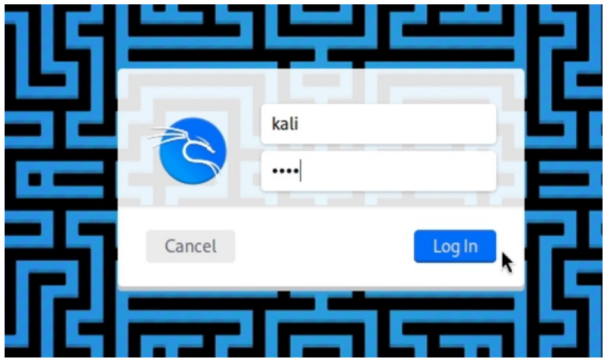
- En la opción de máquinas ya operativas nos encontraremos distintas máquinas preparadas para diferentes entornos de virtualización. En nuestro caso seleccionamos la que está preparada para **VirtualBox**
- Descargar. Es un archivo comprimido con 7zip
- Descargar e instalar 7zip: necesario para descomprimir la máquina virtual. (*)
- Descomprimir el archivo 7zip de la máquina virtual Kali Linux.
- Se genera una carpeta con dos archivos: .vbox y .vdi
- Trasladar esa carpeta a una localización donde se vayan a guardar todas las máquinas virtuales. Por ejemplo, Documentos\MaquinasVirtuales

# Instalando Kali Linux

- Abrir VirtualBox y seleccionar la opción Máquina – Agregar (+)
- Navegar a la carpeta donde tenemos el Kali Linux y dentro seleccionar el archivo .vbox
- Nos debe aparecer la máquina en VirtualBox y ya podemos arrancarla
- Cuando nos pida usuario y contraseña usar

User = kali

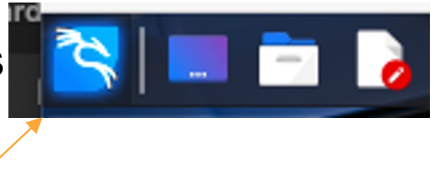
Password = kali



# Configurando Kali

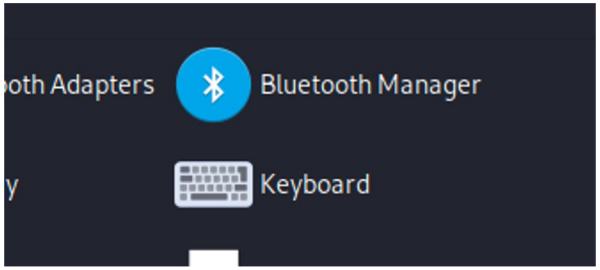
Vamos a realizar algunos ajustes:

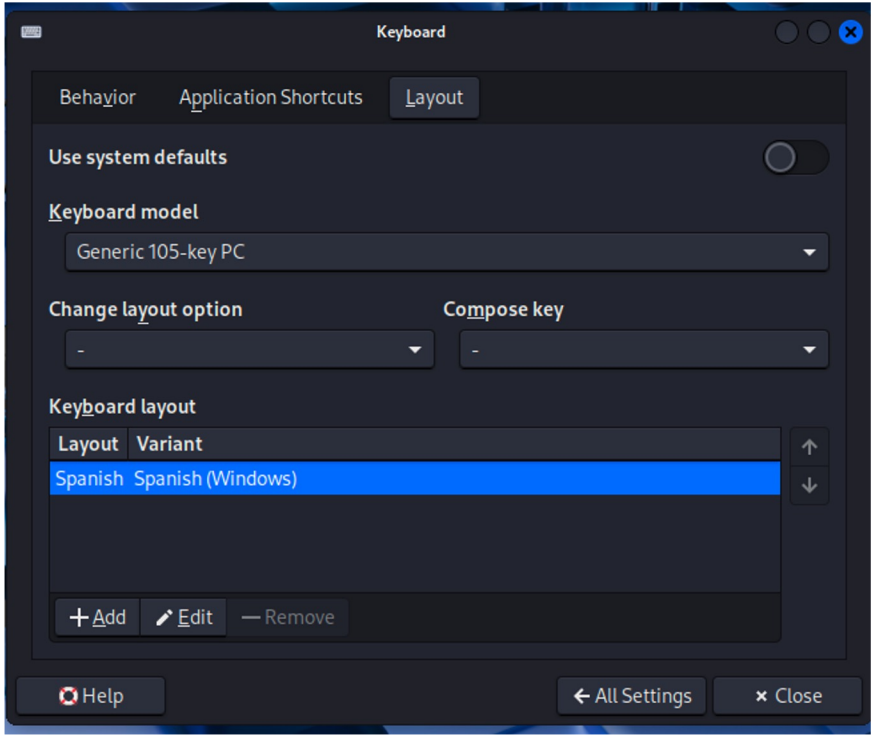
- Kali por defecto tiene el teclado configurado en inglés, vamos a cambiarlo
- Entramos en el menú Applications



- Usamos el buscador de aplicaciones para encontrar Settings Manager y aquí entramos en Keyboard. Seleccionamos la pestaña Layout, quitamos la opción por defecto, añadimos el layout español (Spanish Windows) y eliminamos el inglés

# Configurando Kali



# Configurando Kali

Compartición de archivos:

- Kali nos viene configurado por defecto con la posibilidad de poder transferir ficheros de forma muy cómoda entre el host y la máquina virtual. Simplemente podemos “arrastrar y soltar” de una máquina a otra.

(*)



# Contenido teórico

MEDAC
Instituto Oficial de Formación Profesional

# Seguridad lógica

Su objetivo es salvaguardar la integridad, confidencialidad, la autenticación y disponibilidad de la información almacenada y procesada en los sistemas. Componentes clave:

- **Criptografía:** se encarga de proteger los datos mediante técnicas de cifrado. Existen diferentes tipos de criptografía, como el cifrado de clave secreta (simétrica o privada) y el cifrado de clave pública (asimétrica).

- **Sistemas de identificación:** permiten autenticar y verificar la identidad de los usuarios. Ejemplos serían la firma electrónica o los certificados digitales que desempeñan un papel crucial al garantizar la autenticidad de los organismos y facilitar el intercambio seguro de información.

- **Políticas de contraseñas** sólidas y seguras: requisitos de longitud, complejidad y periodicidad de cambio

# Seguridad lógica

- **Medios de almacenamiento externo seguros:** es importante implementar políticas adecuadas y utilizar medios de almacenamiento externo seguros, como DAS (Direct Attached Storage), NAS (Network Attached Storage) y SAN (Storage Area Network).

- **Copias de seguridad:** la realización de copias de seguridad y la creación de imágenes de respaldo son prácticas esenciales para garantizar la disponibilidad de los datos en caso de fallos o incidentes.

- **Sistemas para asegurar la integridad:** se encargan de preservar los datos para que estos no puedan se modificados. Aquí podemos englobar las políticas de control de permisos de acceso y funciones hash

# Criptografía



## Ejemplo Sencillo

- Cuando un usuario A le envía información a otro usuario B, debe asegurarse de que esa información **no** es alterada o manipulada por el camino (Integridad de la información transmitida).

- La información es para B, por tanto, nadie más debe entender el mensaje (Confidencialidad).

- Debe haber alguna indicación de que el mensaje proviene de A, esto es, debe haber alguna prueba de ello (Autenticación). La autenticación de que el mensaje proviene de A es proporcionada por lo que se denomina firma digital.

- ¿Quién garantiza la identidad y la firma de A? Esto se logra mediante un certificado digital, que autentica la firma de A.

# Encriptación de la información



- La encriptación o cifrado de la información es el proceso por el cual la información o los datos a proteger son traducidos o codificados como algo que parece aleatorio y que no tiene ningún significado (datos encriptados).

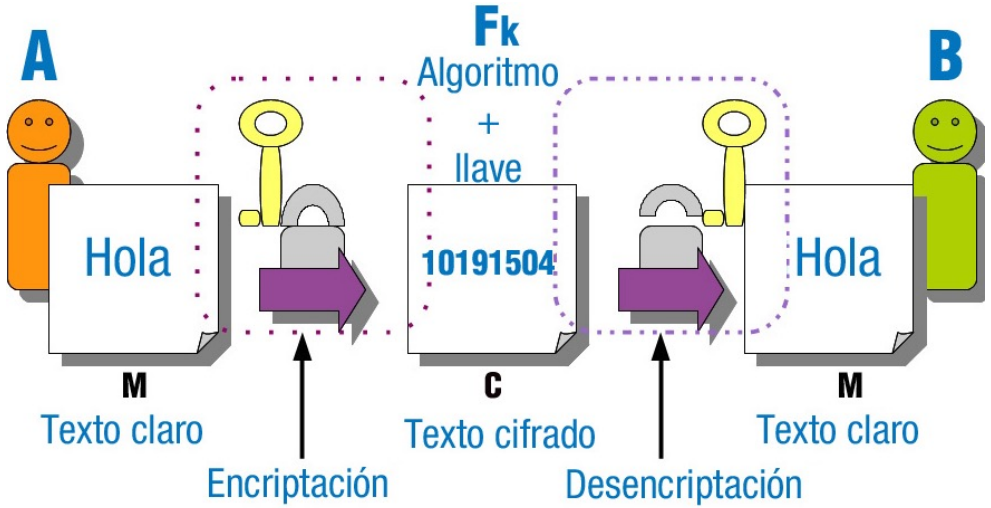
- La desencriptación o descifrado es el proceso inverso, en el cual los datos encriptados son convertidos nuevamente a su forma original.

- Para llevar a cabo tanto la encriptación como la desencriptación se usa un **algoritmo de cifrado** que consiste en un conjunto de pasos u operaciones, normalmente una función matemática. Asociado al algoritmo hay una clave o llave (un número, palabra, frase, o contraseña).

# Encriptación de la información

$$\text{Cifrado} \rightarrow F_K(M)=C$$

El cifrado consiste en aplicar un algoritmo **F**, controlado por una llave **K**, sobre un mensaje o texto claro **M**, para obtener un criptograma o texto cifrado **C**. El descifrado, será el proceso inverso.



# Encriptación de la información

En este ejemplo podrían ser:

- Algoritmo: posición de la letra en el alfabeto representada con dos dígitos.
- Clave: 3.
- Algoritmo Cifrado: posición de la letra en alfabeto (dos dígitos) + 3.
- Descifrado: posición de la letra en alfabeto (dos dígitos) - 3.

Así tendríamos que: H: 7 + 3 → 10, o: 16+3 → 19, l: 12+3 → 15, a: 1+3 → 04.

# Criptografía simétrica o de clave privada

La criptografía simétrica, también conocida como cifrado de clave secreta o privada, utiliza una única clave tanto para el **cifrado** como para el **descifrado** de los datos. La clave se comparte entre el remitente y el receptor.




El algoritmo de cifrado toma la clave y la información original, y realiza una serie de operaciones matemáticas para convertir los datos en un formato cifrado. Luego, la clave se utiliza nuevamente para descifrar los datos cifrados y restaurarlos a su forma original.

# Criptografía simétrica o de clave privada

Ventaja: El cifrado de clave secreta es rápido y eficiente. No aumentan el tamaño del mensaje; por tanto, adecuados para cifrar grandes volúmenes de datos.

Desventaja: Requiere que las partes involucradas compartan y mantengan segura la misma clave. Por tanto, el problema es la distribución de claves

# Algoritmos simétricos

Los principales algoritmos simétricos que se utilizan son DES, AES o ChaCha20

- **DES**: Uno de los primeros algoritmos de cifrado simétrico; en la actualidad ha caído su utilización debido a que se considera inseguro para muchos propósitos, ya que tiene una clave muy corta (es de 56 bits), lo que provoca que se pueda romper relativamente rápido.

- **AES**: Algoritmo de encriptación utilizado en la actualidad, tanto para software como para hardware. Se trata de un algoritmo de codificación por bloques (con un tamaño de bloque fijo de 128 bits). La clave sí presentará un tamaño variable (128, 192 o 256 bits). El funcionamiento de este algoritmo se basa en la construcción de matrices de 4x4 sobre las que se aplican varias rondas de cifrado (transposición, sustitución y mezcla)

# Algoritmos simétricos

- **ChaCha20:** Este algoritmo utiliza cifrado de flujo; es el algoritmo empleado en casi todas las conexiones HTTPS o SSH. El proceso de cifrado utiliza la clave secreta junto con otros valores para generar una secuencia aleatoria (flujo de cables) que después se combina con el mensaje aplicándose bit a bit para realizar el cifrado. Realiza 20 rondas de operaciones matemáticas (sumas, XOR, rotación de bits…)

# Criptografía asimétrica o de clave publica



- La criptografía de clave pública surge para solucionar el problema de distribución de claves que plantea la criptografía de clave privada, permitiendo que emisor y receptor puedan acordar una clave en común sobre canales inseguros. Se la denomina también criptografía asimétrica porque las claves utilizadas para la encriptación y desencriptación son diferentes.

- Utiliza un par de claves relacionadas matemáticamente: una clave pública y una clave privada.

- La clave pública se comparte libremente y se utiliza para cifrar los datos

- Una vez que los datos están cifrados con la clave pública, solo la clave privada correspondiente puede descifrarlos.

# Criptografía asimétrica o de clave publica




cuando A le envía información a B, utiliza la clave pública de B para cifrar el mensaje, que solo podrá descifrar B con su clave privada.

# Criptografía asimétrica o de clave publica



- Ventaja: No existe el problema de distribución de claves, ya que cada parte posee su juego de claves.
- Desventaja: Son más lentos que los algoritmos simétricos.

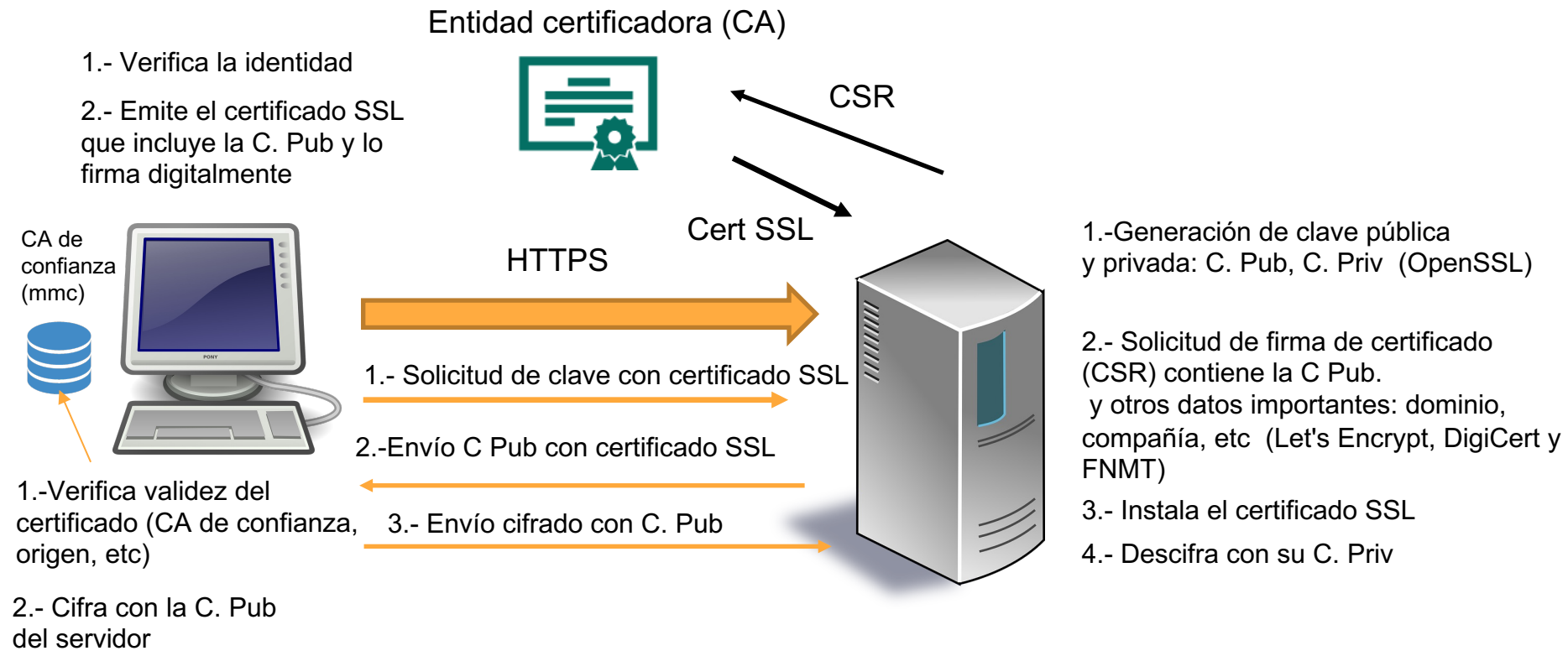
# Algoritmos asimétricos



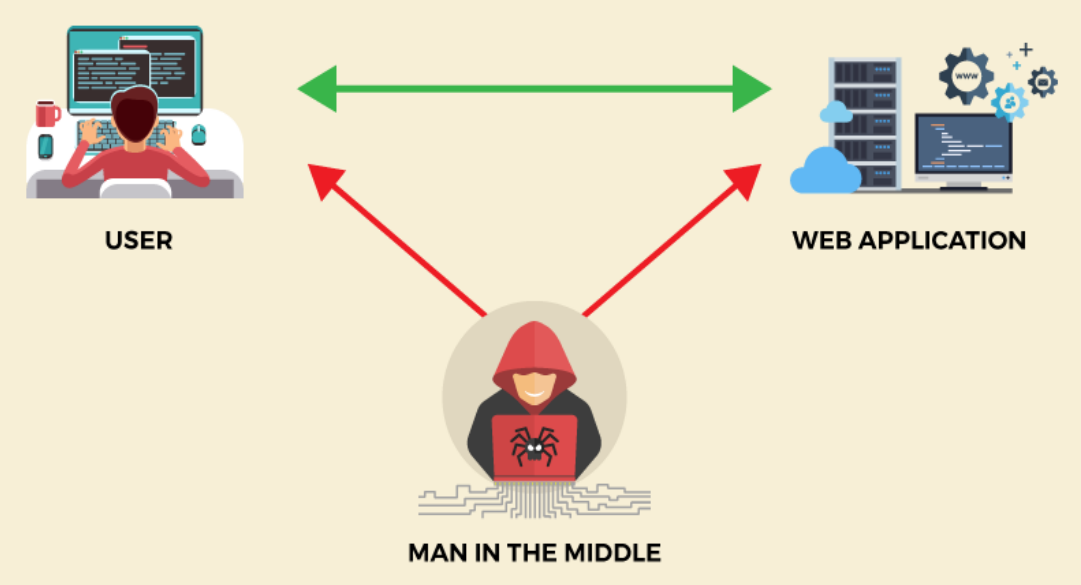
- **RSA** (Rivest, Shamir y Adleman): Uno de los algoritmos de clave pública más presentes en la actualidad. El funcionamiento de este algoritmo se basa en que el destinatario del mensaje genera una clave pública y otra privada; después, el mensaje se cifra usando la clave pública. Para descifrar el mensaje, el destinatario usará su clave privada.

- **DSA** (Digital Signature Algorithm, Algoritmo de Firma Digital): Algoritmo asimétrico que implica el uso de firma digital, por lo que supone un incremento de la seguridad con respecto a RSA.

# Criptografía (Claves Públicas y Privadas - PKI)



# Man in the Middle



# Man in the Middle



Certificados falsos o no confiables:

- Ataque con un certificado falsificado: Si un atacante consigue que el cliente acepte un certificado SSL falso o no confiable, podría hacerse pasar por el servidor legítimo. Esto podría ocurrir si el atacante logra generar un certificado SSL que parece legítimo, pero que no ha sido firmado por una Autoridad de Certificación (CA) confiable.

- Certificados emitidos por una CA comprometida: Si una CA de confianza es comprometida por un atacante, podrían emitirse certificados fraudulentos para suplantar servidores legítimos.

# Man in the Middle



Ataques utilizando certificados autofirmados:

- En algunos casos, los atacantes pueden intentar usar un certificado autofirmado (es decir, uno que no ha sido validado por una CA). Si el cliente no verifica adecuadamente la validez del certificado o decide ignorar advertencias del navegador sobre un certificado no confiable, el atacante podría realizar el ataque con éxito.

# Man in the Middle



Ataques DNS Spoofing:

- Incluso si un sitio web utiliza certificados SSL, un atacante podría llevar a cabo un ataque de spoofing DNS, redirigiendo al cliente hacia un servidor malicioso que usa un certificado válido pero para un dominio diferente. En este caso, si el usuario no revisa que el dominio es correcto, podría estar intercambiando datos con el atacante, aunque la conexión esté cifrada.

- Sin embargo, la advertencia que generalmente muestra el navegador en estos casos ayuda a mitigar este ataque, ya que el dominio del certificado no coincidiría con el dominio solicitado por el cliente.

# Man in the Middle



Ataques con CAs internas o locales:

- En algunos entornos controlados por un atacante (por ejemplo, una red Wi-Fi pública manipulada o un dispositivo infectado), el atacante podría instalar una CA local no autorizada en el dispositivo del cliente o en la red. En este caso, el atacante podría emitir certificados SSL válidos para cualquier sitio web, realizando un ataque sin que el cliente lo detecte fácilmente.

- Este tipo de ataques pueden darse en redes inseguras como puntos de acceso Wi-Fi públicos maliciosos o redes corporativas comprometidas.



# Practicando Criptografía

MEDAC
Instituto Oficial de Formación Profesional

# Generación de claves y encriptación con GPG

- GnuPG es una herramienta de seguridad en comunicaciones electrónicas
- Nos permite realizar funciones básicas como generar un par de claves, intercambiar y comprobar la autenticidad de claves, cifrar y descifrar documentos, y firmar documentos y verificar firmas digitales.
- Veamos su uso en Kali Linux (Linux en general)
- Página oficial: https://gnupg.org/index.html

Nota: Para Mac existe GPG Suite https://gpgtools.org/
Para Windows (Gpg4win) : https://gpg4win.org/download.html

# GPG para encriptación simétrica

- **Encriptar un archivo.** En Kali abrimos el terminal y usamos este comando

```
gpg –c Archivo
```

Se genera un nuevo archivo con extensión gpg que será el fichero encriptado

- **Desencriptar un archivo:**

```
gpg –d Archivo.gpg
```

Para desencriptar en Windows (**Gpg4win**) : https://gnupg.org/download/ (*)

# GPG para encriptación asimétrica

GPG también se puede usar para criptografía asimétrica

- **Listar las claves que tenemos actualmente.** En Kali abrimos el terminal y usamos este comando

```
gpg --list-keys
```

- **Generar par de claves (pública y privada)**

```
gpg --full-gen-key
```

  - Seleccionamos el tipo de clave, longitud, el tiempo de expiración
  - Introducimos un ID de usuario para identificar la clave

(*)

# GPG para encriptación asimétrica



- **Exportar clave pública.** Para poder enviar una clave pública a un interlocutor, antes hay que exportarla. Es necesario indicar el ID de la clave pública

```
gpg --armor --export tu_id_clave > nombre_archivo.asc
```

Para confirmar que la clave se exportó correctamente el fichero generado debería comenzar con -----BEGIN PGP PUBLIC KEY BLOCK----- y terminar con -----END PGP PUBLIC KEY BLOCK-----

Cuando el interlocutor tenga esta clave pública ya podrá enviarte comunicaciones cifradas con esa clave que solo tu podrás descifrar.

(*)

# GPG para encriptación asimétrica

- **Exportar clave privada.** Puedes también exportar tu clave privada si quieres almacenarla en un sitio seguro, pero recuerda no publicarla ni enviarla a nadie

```
gpg --armor –export-secret-keys tu_id_clave > nombre_archivo.asc
```

Para confirmar que la clave se exportó correctamente el fichero generado debería comenzar con -----BEGIN PGP PRIVATE KEY BLOCK----- y terminar con -----END PGP PRIVATE KEY BLOCK-----

(*)

# GPG para encriptación asimétrica



- **Importar clave pública.** Para poder enviar un mensaje cifrado a un interlocutor debes recibir su clave pública, importarla y después usarla para cifrar los mensajes.

```
gpg --import <ruta_del_archivo>
```

Una vez importada pasará a formar parte de nuestro repositorio de claves y la podremos usar para cifrar un mensaje y enviarlo al propietario de esa clave, pero antes necesitamos VALIDARLA

(*)

# GPG para encriptación asimétrica

- **Validar la clave pública importada.** Cuando importas una clave pública, GPG no tiene forma de saber si esa clave fue creada por la persona real o si alguien la ha suplantado. Para resolver este problema vamos a verificar la huella digital (fingerprint) de la clave y le vamos a pedir confirmación a nuestro interlocutor por un canal seguro (teléfono o en persona)

```
gpg --list-keys
```

La fingerprint es el número largo que aparece tras pub rsaxxx fecha

```
pub   rsa3072 2026-10-06 [SC]
      D866C06E39955853D0055458B08ABE4ECF1D1FAE
```

(*)

# GPG para encriptación asimétrica

- **Firma de clave pública importada.** Una vez comprobada que es autentica vamos a firmar la clave y así la dejamos marcada como verificada

```
gpg --edit-key id_clave

gpg> sign
gpg> quit
```

Podemos comprobar que ha quedado firmada con

```
gpg --edit-key id_clave

gpg> check
gpg> quit
```

(*)

# GPG para encriptación asimétrica

- **Encriptar con la clave pública del usuario destino.** Vamos ahora a generar un fichero y encriptarlo con la clave pública de nuestro interlocutor (la que hemos verificado ya)

```
gpg –e –r id_clave nombreFichero
```

Esto genera un fichero encriptado nombreFichero.gpg

Este fichero solo se puede desencriptar con la clave privada de nuestro interlocutor. Si nosotros intentamos desencriptarlo no podremos

```
gpg –d nombreFichero.gpg
```

(*)

# GPG para encriptación asimétrica

- **Desencriptar archivo**: Ahora enviaremos el archivo a nuestro interlocutar que no tendrá problemas en desencriptarlo ya que él si tiene la clave privada correspondiente

```
gpg –d nombreFichero.gpg
```

(*)

# Flujo Completo GPG y claves públicas



# KeyServer en GPG



- Un keyserver (servidor de llaves) es un servicio en Internet que almacena y comparte claves públicas de GPG.

- Su función principal es que otras personas puedan buscar tu clave pública usando tu correo electrónico o el identificador de clave, y así puedan enviarte mensajes o archivos cifrados que solo tú podrás descifrar con tu clave privada.

- Existen varios keyservers populares como: keyserver Ubuntu, Opengpg, etc.

- En la práctica que realizaremos en este tema se pide investigar como subir una clave pública al keyserver Ubuntu para que otros usuarios la puedan localizar a través de tu correo electrónico y puedan usarla para mandarte archivos encriptados



# Contenido teórico

MEDAC
Instituto Oficial de Formación Profesional

# Funciones mezcla o resumen (hash)



- Las funciones de mezcla o resumen se utilizan para verificar la integridad de los datos y garantizar que no hayan sido modificados durante la transmisión o el almacenamiento.

- Estas funciones aplican algoritmos matemáticos que transforman los datos en una cadena de longitud fija, también llamada hash o resumen

- Las funciones hash son unidireccionales, lo que significa que no se pueden recuperar los datos originales a partir del hash

- Una pequeña alteración en los datos originales resultará en un hash completamente diferente.

# Algoritmos de hashing

- **MD5**: El algoritmo MD5 genera un hash de 128 bits (32 caracteres en hexadecimal). Fue uno de los métodos de hashing más populares, especialmente en la década de 1990. Es rápido y fácil de implementar, pero es vulnerable a ataques de colisión (cuando dos entradas diferentes producen el mismo hash), por lo que no se considera seguro para aplicaciones de seguridad críticas.



# Algoritmos de hashing

- **SHA-1 :** SHA-1 produce un hash de 160 bits (40 caracteres en hexadecimal). Fue ampliamente utilizado y ofreció mayor seguridad que MD5 durante mucho tiempo. Finalmente se descubrió que también es vulnerable a ataques de colisión, y su uso ha disminuido en aplicaciones de alta seguridad.

- **SHA-2**: SHA-2 es una familia de funciones hash que incluye SHA-224, SHA-256, SHA-384 y SHA-512. El número indica el tamaño del hash en bits, siendo SHA-256 (256 bits) uno de los más utilizados actualmente. Es uno de los algoritmos más seguros y es el estándar actual en la mayoría de sistemas modernos, pero requiere mayor potencia de procesamiento en comparación con algoritmos más antiguos.

# Algoritmos de hashing

- **SHA-3:** SHA-3 es una familia de funciones hash desarrollada como alternativa a SHA-2 y utiliza un enfoque diferente. Ofrece alta seguridad y se diseñó para resistir a futuros avances en ataques de colisión. No se ha implementado tan extensamente como SHA-2, aunque su uso está creciendo en sistemas que requieren un mayor nivel de seguridad.



# Practicando Hashing

MEDAC
Instituto Oficial de Formación Profesional

# Uso de MD5 en Linux – md5sum



- Linux (Kali) tiene una herramienta que permite calcular el hash de un fichero o conjunto de ficheros. También nos permite comprobar sin un fichero tiene un cierto hash para verificar si ha sido o no modificado. La herramienta es **md5sum**

- **Generar el hash de un documento:**

```
md5sum FicheroPrueba
```

Si quiero guardar el hash resultante en un archivo

```
md5sum FicheroPrueba > hashFicheroPrueba.md5
```

(*)

# Uso de MD5 en Linux – md5sum

El resultado de la generación del hash es un fichero que contiene esta línea

```
b08a17ce7a11230aca48f28679332473  FicheroPrueba
```

- **Verificar un hash:** Podemos verificar que el hash para ese fichero es correcto usando este comando

```
md5sum –c hashFicheroPrueba.md5
```

md5sum leerá la primera línea y buscará el fichero indicado en el directorio donde estemos ejecutando el comando. Calculará su hash y lo comparará con el almacenado

(*)

# Uso de MD5 en Linux – md5sum

- Si es correcto el resultado será:

```
┌──(kali㉿kali)-[~/Documents]
└─$ md5sum -c FicheroPrueba.md5
FicheroPrueba: OK
```

- Si modificamos el fichero y volvemos a ejecutar el comando de verificación el resultado será

```
┌──(kali㉿kali)-[~/Documents]
└─$ md5sum -c FicheroPrueba.md5
FicheroPrueba: FAILED
md5sum: WARNING: 1 computed checksum did NOT match
```

# Uso de MD5 en Linux – md5sum



- Podemos calcular el hash de varios ficheros dentro de una carpeta de una vez

```
md5sum * > hashCarpeta.md5
```

Realiza el hash completo sobre la carpeta donde se ejecute

- Cualquier modificación en uno de los ficheros hará que falle la verificación del hash

(*)



# Contenido teórico

MEDAC
Instituto Oficial de Formación Profesional

# Sistemas de identificación

Sistemas que se basan en técnicas criptográficas para verificar y garantizar la identidad de los usuarios

## Firma electrónica



La firma electrónica proporciona una forrmia de verificar la autenticidad del documento y aseguar que no ha sido alterado desde su firma

## Certificados digitales



Son documentos electrónicos utilizados para autenticar la identidad de una persona u organismó a través de la red. Son emitidos por autoridades de certificación confiables

## DNI electrónico



El DNI electrónico incorpora un chip inteligente que almacena datos persenales y una clave privada unica aso-ciada a la identidad del titular, Esta clave priyada se utiliza para autenticarı y firmar

# Listas de control de acceso - ACL

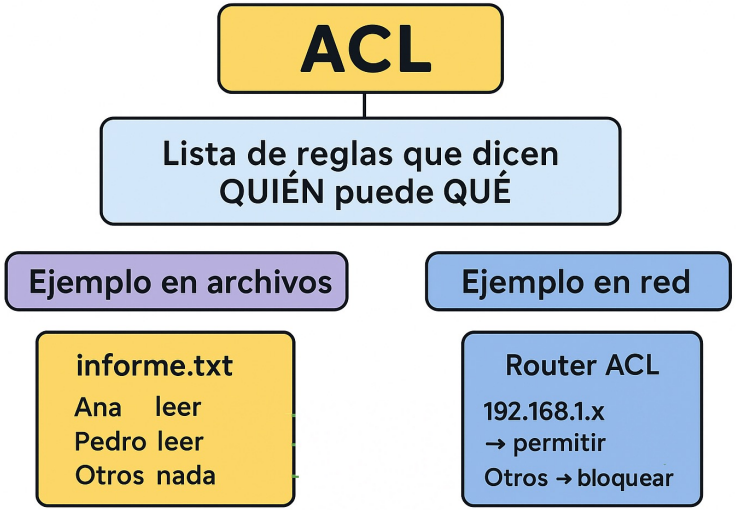


- Son herramientas utilizadas en seguridad informática para controlar y gestionar el acceso a recursos o sistemas.
- Las ACL son reglas que definen quién (usuario, grupo, dirección IP, etc.) puede hacer qué (leer, escribir, ejecutar, acceder, bloquear) sobre un recurso.
- Cuando un usuario o dispositivo intenta acceder a un recurso, el sistema consultará la ACL correspondiente y tomará decisiones basadas en las reglas establecidas.
- Se pueden establecer permisos específicos para usuarios individuales, grupos de usuarios o categorías de usuarios

# Listas de control de acceso - ACL

Se usan en dos ámbitos principalmente:

- **Sistemas operativos** → controlar permisos sobre archivos y carpetas.
- **Redes** → controlar qué tráfico pasa o se bloquea en un router/firewall





# Practicando ACL

MEDAC
Instituto Oficial de Formación Profesional

# Forma de acceder a los permisos Linux

En Kali (línea de comandos) se puede acceder a los permisos de un archivo con

```
ls –l fichero
```

```
mariceli@mariceli-Ubuntu:~$ ls -l FicheroPrueba
-rw-rw-r-- 1 mariceli mariceli 103 sep  9 22:29 FicheroPrueba
```

Significado de las letras
r → read (leer)
w → write (escribir)
x → execute (ejecutar)
- → permiso no concedido

3 bloques correspondientes cada uno a:
- Propietario
- Grupo
- Otros

(*)

# Cambiar los permisos Linux

Si somos los propietarios de un archivo o administradores del sistema podemos cambiar los permisos de un archivo. Para ello se usa el comando `chmod` y se tiene en cuenta esta equivalencia:

r = 4, w = 2, x = 1

Cuando establezcamos que nuevos permisos queremos para propietario, grupo y otros sumamos los valores correspondientes a cada grupo.

Ejemplo: Si quiero 


= 644 (dueño rw- = 6, grupo r-- = 4, otros r-- = 4)

```
chmod 644 fichero
```

(*)

Davante